

CSC 725 Homework 2: Implementing a Three-Layer Convolutional Neural Network

Objective

Construct, train, and evaluate a three-layer convolutional neural network (CNN) in PyTorch for image classification on the CIFAR-10 dataset.

Requirements

1. Setup (10 points)

- Load the CIFAR-10 dataset using `torchvision.datasets`.
- Use the CIFAR-10 training set for model training and the test set for final evaluation.
- Use `torch.utils.data.DataLoader` to create mini-batches for training and testing.
- Shuffle the training data during training.

2. Model Architecture (30 points)

Implement a CNN with the following architecture:

- **First Convolutional Block**
 - Convolutional layer with 16 output channels, a 3×3 kernel, stride 1, and padding 1.
 - ReLU activation.
 - Max-pooling layer with a 2×2 kernel and stride 2.
- **Second Convolutional Block**
 - Convolutional layer with 32 output channels, a 3×3 kernel, stride 1, and padding 1.
 - ReLU activation.
 - Max-pooling layer with a 2×2 kernel and stride 2.
- **Third Convolutional Block**
 - Convolutional layer with 64 output channels, a 3×3 kernel, stride 1, and padding 1.
 - ReLU activation.
 - Max-pooling layer with a 2×2 kernel and stride 2.
- **Output Layer**
 - Flatten the output of the final max-pooling layer.
 - Use a fully connected layer to map the flattened representation to 10 output classes corresponding to the CIFAR-10 categories.

3. Training (20 points)

- Use cross-entropy loss as the objective function.
- Use the Adam optimizer.
- Train the model for at least 10 epochs.
- After each epoch, report the training loss and training accuracy.

- Record the training loss and accuracy so that they can be included in the final report.

4. Testing (10 points)

- After training is complete, evaluate the model on the CIFAR-10 test set.
- Report the final test accuracy.

5. Report (30 points)

Submit a brief report containing the following:

- A description of the final CNN architecture.
- Documentation of the training process, including training loss and accuracy across epochs.
- Graphs of the training loss and training accuracy.
- A discussion of how the convolutional, activation, pooling, and fully connected layers contribute to the learning process and overall model performance.
- A discussion of any implementation or training challenges encountered and how they were addressed.

6. Extra Credit: Model Complexity Analysis (up to 20 points)

Implement functions in PyTorch to automatically calculate the number of learnable parameters and the computational cost of your CNN model.

- **Parameter Counting (8 points)**

Implement a function

```
count_parameters(model)
```

that calculates the total number of learnable parameters in the model.

Your function should:

- Compute the number of parameters in each convolutional and fully connected layer.
- Include both weights and bias parameters.
- Report the number of parameters for each layer.
- Report the total number of learnable parameters in the entire model.

- **FLOPs Calculation (8 points)**

Implement a function

```
calculate_flops(model, input_size)
```

that estimates the total number of floating-point operations (FLOPs) required for one forward pass through the model.

For this assignment, count one multiplication as one FLOP and ignore additions and bias operations.

For a convolutional layer, use

$$FLOPs = H_{out} \times W_{out} \times C_{out} \times C_{in} \times K_H \times K_W.$$

For a fully connected layer, use

$$FLOPs = D_{in} \times D_{out}.$$

Your function should:

- Determine the output dimensions of each convolutional layer.
 - Calculate the FLOPs for each convolutional layer.
 - Calculate the FLOPs for the fully connected layer.
 - Report the FLOPs for each layer.
 - Report the total FLOPs required for one forward pass.
 - Ignore the computational cost of ReLU, max-pooling, and bias operations.
- **Results and Verification (4 points)**
Include the following in your report:
 - The parameter count for each layer and the total number of parameters.
 - The FLOPs for each layer and the total FLOPs for one forward pass.
 - A brief explanation of how your functions perform these calculations.
 - Manually verify the parameter count and FLOPs of at least one convolutional layer and compare the results with the outputs of your functions.
 - **Important:** You should implement these calculations yourself. Do not use external model-profiling libraries such as `torchinfo`, `thop`, or similar packages to directly obtain the answers.

Hints and Tips

- Make sure that the training data are shuffled during training.
- Use `torch.utils.data.DataLoader` to efficiently create mini-batches.
- Carefully determine the dimensionality of the feature maps after the final max-pooling layer before defining the fully connected layer.
- Make sure that the model is placed in the appropriate training or evaluation mode using `model.train()` and `model.eval()`.
- If a GPU is available, you may use it to accelerate training.